Estáticos

Antes de empezar a definir el concepto de estáticos tenemos que entender que existen dos tiempos en el proceso de compilación de un programa. Existe el tiempo de compilación y el tiempo de ejecución. En programación, el concepto de tiempo de compilación y tiempo de ejecución se refiere a cuándo se toman ciertas decisiones o cuándo ocurren ciertas acciones en el ciclo de vida de un programa. En el caso de las variables estáticas en C y C++, estos conceptos tienen implicaciones específicas:

Tiempo de compilación

El tiempo de compilación se refiere al momento en que el código fuente se traduce a un lenguaje de bajo nivel, como código máquina, por parte del compilador. En este proceso, se verifican las declaraciones, la asignación de memoria, los tipos de datos y otros aspectos del código.

Tiempo de ejecución

El tiempo de ejecución se refiere al momento en que el programa ya está en funcionamiento y el código está siendo ejecutado por el procesador. Aquí es cuando ocurren las operaciones dinámicas, como la asignación de memoria en el heap, el manejo de eventos, y, en el caso de las variables, las operaciones sobre los valores asignados.

Características

Cuando hablamos de un miembro estático decimos que este miembro se define en tiempo de compilación, por lo tanto, toma sus valores antes que se ejecute el programa. Cuando se declara una variable estática, el compilador reserva una dirección de memoria específica para ella. A diferencia de las variables locales automáticas, que se crean y destruyen cada vez que una función se llama, las variables estáticas se asignan en tiempo de compilación y persisten en la memoria durante toda la vida del programa.

Para determinar un miembro estático utilizamos la palabra reservada ***static*** antes de la declaración del atributo o método.

Todo miembro estático puede acceder a cualquier miembro estático, pero no puede acceder a miembros dinámicos (tiempo de ejecución) ya que estos elementos se crean después que los miembros estáticos. Por el contrario, cualquier miembro dinámico o de tiempo de ejecución puede acceder sin problemas a miembros estáticos.

Los miembros estáticos tienen la particularidad que son independientes de las instancias de los objetos. Las instancias se realizan en tiempo de ejecución mientras que los miembros estáticos se definen antes. Teniendo esto en cuenta podemos pensar que un miembro estático funciona a modo de miembro global y nos permite acceder a ellos directamente desde la clase, no se necesita invocarlos desde un objeto instanciado.

Un ejemplo muy claro de esto, podría ser una variable estática que nos cuente las cantidades de instancias que existen de un objeto determinado. La variable que manipula este dato la declaramos como estática, y dentro del constructor aumentamos en una unidad esta variable; por el contrario, en el destructor disminuimos esta variable. Este dato lo podemos ir verificando en cualquier momento desde la propia clase. Veamos el ejemplo:

```
class instancias
{
private:
    static int cantidad;
public:
    instancias();
};

int instancias::cantidad=0;

instancias::instancias()
{
    this->cantidad++;
}
```

En el tiempo de compilación se crea la variable cantidad inicializada en 0. Luego en el constructor cada vez que se instancia un objeto aumentamos el valor de esta variable. Vamos a agregar a este código un get estático para leer el valor y un destructor que disminuya la cantidad de instancias:

```
class instancias
{
private:
    static int cantidad;
public:
    instancias();
    static int get_instancias(){return cantidad;}
    ~instancias(){cantidad--;}
};

int instancias::cantidad=0;

instancias::instancias()
{
    this->cantidad++;
}
```

Ahora armemos un main para verificar estas características:

```
int main()
{
    cout<<"Cantidad de instancias: "<<instancias::get_instancias()<<endl;
    instancias *i0=new instancias();
    instancias *i1=new instancias();
    instancias *i2=new instancias();
    cout<<"Cantidad de instancias: "<<instancias::get_instancias()<<endl;
    delete i0;
    cout<<"Cantidad de instancias: "<<instancias::get_instancias()<<endl;
    return 0;
}
```

Los métodos y atributos estáticos los podemos acceder directamente desde la clase, también existe la posibilidad de acceder a ellos desde la instancia de un objeto.

Por consola vemos el siguiente resultado:

```
Cantidad de instancias: 0
Cantidad de instancias: 3
Cantidad de instancias: 2
```